

Traversal-Aware Structural Fusion of AST and CFG for Code Representation Learning in Software Fault and Code Smell Detection

I. IMPLEMENTATION AND HYPERPARAMETER DETAILS

The methods and techniques presented in this work are implemented in Python 3.10 using a combination of machine learning, deep learning, graph processing, and program analysis libraries. The classical machine learning algorithms, such as Random Forest classifiers, are implemented with the *Scikit-learn* library, whereas neural models and deep learning are built with *PyTorch* and *TensorFlow*, along with pre-trained transformers such as CodeBERT from the Huggingface Transformers library.

Static program analysis is performed through the construction of Abstract Syntax Trees using Java parsers based on *Eclipse JDT* and *JavaParser*. Control flow graphs are extracted using static analysis scripts and pipeline construction. Graph processing and traversing are performed using *NetworkX* and custom traversal libraries to generate Root-to-Leaf (RL) and Leaf-to-Leaf (LL) path contexts. The implementation also includes utilities for:

- AST and CFG traversal and serialization,
- Path-context extraction,
- embedding generation,
- dataset preprocessing and normalization,
- feature fusion for AST+CFG representations,
- model training and evaluation,
- and experimental result aggregation.

The source code goes through a pipeline that starts with parsing the Java source code, resulting in the transformation of every source file into an AST and a CFG. In the case of AST, paths are constructed for both root-to-leaf (RL) and leaf-to-leaf (LL) paths. Graph-based paths are created using CFGs. The extracted path contexts are then embedded using either:

- 1) Traditional structural embeddings (Code2Vec-style path embeddings), or
- 2) Transformer-based semantic embeddings using CodeBERT.

In AST+CFG embeddings, both types of embeddings are combined prior to classification through concatenation or fusion.

The experimental framework supports both traditional machine learning and neural representation-based classification pipelines.

Charith Kutikuppala and Saurabh Tiwari are with Software Engineering Research Lab, DA-IICT, Gandhinagar, Gujarat, India. Santosh Singh Rathore is with the Department of Computer Science and Engineering, ABV-Indian Institute of Information Technology and Management Gwalior, India. E-mail: 202101128@daict.ac.in, *saurabh_t@daict.ac.in, santosh@iiitm.ac.in, *corresponding author

Random Forest Classifier: Random Forest was used as the primary downstream classifier for evaluating handcrafted and embedded representations. Hyperparameters were optimized using grid-search-based tuning with 5-fold cross-validation. The primary parameters include:

- Number of estimators (n_estimators): 100
- Maximum tree depth (max_depth): 20
- Criterion: Gini impurity
- Cross-validation folds: 5

CodeBERT Configuration: The pre-trained model from Hugging Face was used for transformer-based embedding generation. Fine-tuning and embedding extraction were performed with the following settings:

- Learning rate: 1×10^{-5}
- Epochs: 2–3
- Early stopping patience: 1 epoch
- Batch size: 16 (PROMISE datasets)
- Batch size: 32 (MLCQ datasets)
- Embedding dimension: 768

An optional PCA reduction to 256 dimensions was applied before downstream classification when necessary.

Code2Vec Configuration: Code2Vec-based structural embeddings were generated using AST path contexts extracted from Java methods. The implementation follows the original Code2Vec design with the following settings:

- Embedding dimension: 128
- Maximum AST paths per method: 200
- Maximum path length: 8 nodes

To trade off computational efficiency against representation efficiency, path-sampling techniques were adopted for embedding generation.

Data Preprocessing: The tool automatically parses source code files for both the PROMISE and MLCQ corpora. Invalid or unparseable files are filtered out during preprocessing. Data preprocessing tasks, including data balancing, data splitting, normalization, and representation saving, are performed by specialized preprocessing functions.

In cross-version experiments, separate training and test sets were used for each software version. For code smells, different data preprocessing methods were adopted based on the target code smells.

Experimental Infrastructure: Traditional representation extraction and preprocessing experiments were executed on a local workstation equipped with:

- Intel Core i7 processor,
- 16–32 GB RAM,
- and SSD-based storage.

Deep learning and transformer-based experiments were conducted on Google Colab Pro using NVIDIA T4/A100 GPUs and TPU v5e instances. GPU acceleration was primarily used for CodeBERT embedding extraction and fine-tuning tasks.

To ensure reproducibility, all experiments used deterministic random seeds for dataset splitting, model initialization, and path sampling. The same hyperparameters, preprocessing steps, and evaluation scripts were used for all the experiments.